

串連系統 v2.0

管理層決策輔助系統 · 演算法與資料邏輯文件 (v2)

本文件說明系統使用的核心演算法、評分公式、決策狀態判定、組織健康度模型、決策成效追蹤（Decision Impact）、What-if 決策模擬器、以及近期 bug 修復記錄。適合教師檢閱、新進工程師交接、論文撰寫附錄。

專案名稱	串連系統 v2.0 (Chuanlien System)
應用場景	20 – 50 人規模投資公司的管理層決策輔助
前端技術	React 19 + TypeScript + Vite 6 + Tailwind v4 + Framer Motion + Recharts
後端服務	Firebase Authentication + Cloud Firestore
核心演算法	BM25F、TF-IDF、Gini Coefficient、Empirical Percentile、Weighted Load Model、Local Minima Detection、Decision Impact、What-if Simulation
最新功能	asOf 時間切片、決策狀態動態判定、Decision Impact 追蹤、What-if 模擬器
作者	資管導論 第 13 組
文件版本	v2.1 · 2026-05

目錄

1. 系統總覽與核心模組
2. 加權員工負載模型（含 asOf 時間切片）
3. 卡點分位數風險分析（Empirical Percentile）
4. BM25F 歷史案件搜尋引擎
5. 智能案件推薦（Smart Recommendation）
6. 部門互動網絡分析
7. ORI 組織風險指數
8. 組織健康度 6 維雷達 + 12 週趨勢
9. ★ 決策狀態 helper（動態判定）
10. ★ Decision Impact 決策成效追蹤
11. ★ What-if 決策模擬器
12. 資料同步機制與保護邏輯
13. Bug 修復記錄（v2.0 → v2.1）
14. 結語：方法論選擇背後的设计取舍

1. 系統總覽與核心模組

串連系統 v2.0 是面向中小型投資公司管理層的決策輔助平台，整合週報、案件交接、決策追蹤、員工負載、卡點預警、歷史檢索、組織網絡、會議準備、What-if 模擬、Decision Impact 追蹤等 11 個模組。系統的設計核心並非「資料庫」，而是「演算法層」 — 在原始資料之上，用統計、資訊檢索、圖論方法把瑣碎事件融合成「組織儀表板」與「決策閉環」。

1.1 模組與對應演算法

週報填寫	Markdown 格式 + 關鍵字 tokenize
案件交接	簽收狀態機 + 逾時 SLA 監控
決策追蹤	★ 動態狀態判定 (dueDate vs asOf) + Decision Impact 追蹤
員工負載	★ 加權負載模型 + Gini 係數 + 12 週 asOf 切片
歷史搜尋	BM25F + 多 n-gram + 同義詞 + Substring Boost
卡點分析	★ Empirical Percentile (含 asOf 時點)
組織分析	雙向 Adjacency Matrix + 力導向圖 + 單向溝通偵測
會議準備	事件聚合 + 風險排序
組織健康	6 維加權雷達 + 12 週時間序列 + Local Minima 拐點 + 事件 inline 展開
智能推薦	BM25F + 同類別過濾 + 解決天數中位數
What-if 模擬	★ 場景修改 + 模擬快照對比 + 維度 delta 顯示

★ = v2.1 新增或大幅升級

2. 加權員工負載模型（含 asOf 時間切片）

本系統不採用「處理幾件案件」這種絕對計數，而是 加權負載分數（Weighted Load Score）融合四個訊號，並依時間衰減：

2.1 計算公式

$$\text{loadScore} = \text{timeWeightedCases} \times 1.5 + \text{blockerLoad} \times 2.0 + \text{mentionsWeighted} \times 0.8 + \text{handoffLoad} \times 1.0$$

2.2 各分量說明

timeWeightedCases	員工被指派的案件數，依離 asOf 多遠做指數衰減。本週權重 1.0、1 週前 0.7、2 週前 0.5……故近期高強度工作會被放大。
blockerLoad	員工目前負責的活躍卡點數 $\times 2.0$ 加權，因為卡點代表正在燒的工作。
mentionsWeighted	員工在他人週報中被提及的次數（共同負責、跨部門協作）。權重 0.8。
handoffLoad	員工作為交接接收者的待簽收件數，含逾時加成。

2.3 asOf 時間切片（v2.1 新增）

原本 analyzeEmployeeLoad 內部用 new Date()，所有快照皆以「現在」為中心，導致 12 週趨勢圖每週數字相同。v2.1 新增 asOf 參數（預設 NOW），讓函式以指定時點作為時間衰減中心，未來資料（asOf 之後的 reports/handoffs）自動排除。

```
analyzeEmployeeLoad(reports, handoffs, employees, asOf = NOW)
  weeksAgo = round((asOfMs - reportWeekMs) / 7 days)
  if reportDate > asOf → ■■
  decay = TIME_DECAY[weeksAgo]
```

2.4 過載判定（Percentile-based）

overload	loadScore ≥ 25 或 percentile ≥ 90
high	loadScore ≥ 15 或 percentile ≥ 75
normal	loadScore ≥ 6
low	loadScore ≥ 1
idle	loadScore < 1 （流動人力儲備）

為什麼用 Percentile 而不是固定門檻？

公司規模不同、產業淡旺季不同，「絕對 25 分」對小公司是過載、對大公司是普通。Percentile 自動適應分布，無論 10 人還是 100 人都能找出「相對最忙的」。

3. 卡點分位數風險分析 (Empirical Percentile)

傳統卡點告警常用「卡了 N 天」這種絕對天數，但不同類別合理時長差異極大。本系統用 Empirical Percentile（經驗分位數），把當前卡點對照同類別歷史的解決天數分布，計算「超過了多少 %」。

3.1 計算步驟

1. 從 SEED_HISTORY（53 筆歷史案件）依 tags 過濾出同類別案例。
2. 提取每筆案例的解決天數（從 outcome 字串「已解決 · N 天」 parse）。
3. 若同類樣本 ≥ 5 筆 → 用同類分布；否則 fallback 用全公司歷史。
4. 計算 P75 / P90 / P95，以及當前卡點所處的 percentile rank。

3.2 asOf 參數 (v2.1 新增)

原本 analyzeBlockerRecord 用 new Date() 算 currentDays，歷史快照永遠用「現在」算，導致歷史週的 P95+/P90+ 卡點被高估。v2.1 新增 asOf 參數， $\text{currentDays} = (\text{asOf} - \text{createdAt}) / 86400000$ ，讓組織健康度的 12 週趨勢線準確反映各週風險。

3.3 風險等級判定

critical（極高風險）	$\text{currentDays} \geq \text{P95} \rightarrow$ 立刻召開協調會議
high（高風險）	$\text{currentDays} \geq \text{P90} \rightarrow$ 建議本週內升級處理
medium（關注中）	$\text{currentDays} \geq \text{P75} \rightarrow$ 進入關注區
normal（正常）	$\text{currentDays} < \text{P75} \rightarrow$ 仍在正常處理時程內

3.4 樣本不足處理

若同類歷史不足 5 筆 → 改用全公司歷史；仍不足 → $\text{hasData} = \text{false}$ ，退化為時間 SLA 提醒（21 天 → high、14 天 → medium）。

4. BM25F 歷史案件搜尋引擎

從 TF-IDF + Cosine 升級為 BM25F (Okapi BM25 with Field Weighting) , Elasticsearch / Lucene 同款。

4.1 BM25 TF 飽和函數

```
tf_normalized = (tf × (k1 + 1)) / (tf + k1 × (1 - b + b × (len / avgLen)))
```

其中 k1 = 1.5 (TF 飽和速度)、b = 0.75 (長度正規化強度)。

4.2 欄位權重

title (標題)	5.0 — 公司名 / 案件主題出現於標題, 相關性遠高於內文
tags (標籤)	4.0
summary (摘要)	2.0
outcome (結論)	1.5
owner (負責人)	1.0
detail (詳細)	1.0

4.3 中文 Tokenization : 多 n-gram 混合

中文沒有明確詞界, 本系統同時切 1-gram、2-gram、3-gram :

「東京中央銀行」→ 東 / 京 / 中 / 央 / 銀 / 行 / 東京 / 京中 / 中央 / 央銀 / 銀行 / 東京中 / 京中央 / 中央銀 / 央銀行。3-gram 特別重要, 因為投資領域多為 3-4 字專有名詞。

4.4 Substring Boost

```
if document.contains(query) → score × 1.8
if title.contains(query) → score × 1.4
```

4.5 同義詞表 (14 組)

募資 ≈ 融資 ≈ 募款 ≈ fundraising ; 盡調 ≈ 盡職調查 ≈ DD ≈ due diligence ; NDA ≈ 保密協議 ; LOI ≈ 意向書 ; 投委會 ≈ 投資委員會 ≈ IC ; 估值 ≈ valuation ≈ 定價 ; 退場 ≈ exit ≈ 出場 ; 法遵 ≈ compliance ; 風控 ≈ 風險管理 ≈ risk ; Pre-A ≈ PreA。

4.6 IDF — Robertson-Sparck-Jones

```
idf(t) = log(1 + (N - df(t) + 0.5) / (df(t) + 0.5))
```

5. 智能案件推薦

卡點 / 交接詳情頁中，系統自動跑 BM25F 找過去 3 筆最相似的歷史案。把「歷史搜尋」從被動工具轉成主動決策輔助。

觸發時機	卡點分析頁展開卡點、交接 Modal 開啟
查詢字串	標題 + 描述 + 類別 + 案件 ID 串接後送入 BM25F
過濾	relevance > 20%、取前 3 筆
展示	相關度 % + 解決天數 + 速度標籤（快/正常/較慢/延誤）+ 平均解決天數

為什麼不用 LLM Embedding？

(1) 投資資料機密，不能送 cloud API。(2) BM25F 可解釋（為什麼推薦這筆？因為標題命中『東京中央銀行』）。(3) 對 50-200 筆歷史案的中小型企業，BM25F 準確度足以匹敵 Embedding，且零成本。

6. 部門互動網絡分析

從週報文字與交接紀錄中萃取「誰提到誰」，建構部門間的有向加權圖。

邊權重來源	(1) 週報 mention：部門 A 週報中提到部門 B 的次數；(2) 交接量：A → B 交接單數
視覺化	Force-directed Graph，節點大小依對外溝通總量
時間範圍	4 週 / 8 週 / 3 月 / 半年 / 1 年 / 全部
單向偵測	若 $\text{weight}(A,B) \geq 5$ AND $\text{weight}(B,A) = 0 \rightarrow$ 標記單向溝通（組織病徵）

7. ORI 組織風險指數

$$\text{ORI} = 0.35 \times \text{HCC} + 0.25 \times \text{DL} + 0.25 \times \text{BT} + 0.15 \times \text{CDC}$$

HCC	Human Capital Concentration · Gini 係數 + top1 佔比 + 離群數
DL	Decision Latency · avgCompletionDays + 逾期數
BT	Blocker Tail Risk · avgPercentile + P90/P95 數
CDC	Cross-Dept Comm · 單向溝通組數

8. 組織健康度 6 維雷達 + 12 週趨勢

ORI (0-200 反向) 對管理層不直覺。v2.1 新增「組織健康度」採 0-100 正向計分, 6 維雷達 + 12 週序列 + 拐點偵測 + 點擊事件 inline 展開。

8.1 整體分數加權

```
overall = blockerHealth × 0.22 + decisionTimeliness × 0.18
          + handoffSmoothness × 0.15 + loadBalance × 0.18
          + crossDept × 0.12 + reportQuality × 0.15
```

8.2 六個維度詳解

卡點健康 (22%)	$100 - P95\text{數} \times 15 - P90\text{數} \times 7 - \max(0, \text{avgPercentile} - 50) \times 0.8$
決策及時 (18%)	$100 - \max(0, \text{avgCompletionDays} - 14) \times 3 - \text{逾期數} \times 10$
交接流暢 (15%)	$50 + \text{完成率} \times 50 - \text{逾時 24h+ 件數} \times 8$
負載均衡 (18%)	$100 - \max(0, \text{Gini} - 0.35) \times 200 - \text{過載員工} \times 8$
部門協作 (12%)	$100 - \text{單向溝通組數} \times 15$
週報品質 (15%)	$\min(1, \text{繳交率}) \times 60 + \text{內容字數} \times 0.3 + \text{卡點欄填寫率} \times 10$

8.3 拐點偵測 (Local Minima)

```
if series[i] < series[i-1] - 3 AND series[i] < series[i+1] - 3 → ■■
```

8.4 事件 chip 點擊 inline 展開 (v2.1 新增)

點趨勢線任一週 → pin 住該週事件框。事件 chip 改為「可點開」(非跳頁), 依 asOf 時點還原當週的實際清單 (含部門 / 負責人 / 當週天數)。右上角保留「前往完整頁面 →」連結。

9. ★ 決策狀態 helper（動態判定）

v2.0 各頁面用 `d.status === "逾期"` 字串判定，但 `orgHealth` 用動態日期判定，造成兩套標準並存：一筆 `status` 仍是「執行中」但 `dueDate` 已過的決策，`orgHealth` 會算逾期扣分（事件提示「N 筆決策逾期」），但 `Dashboard / Decisions` 頁卻找不到。v2.1 新增 4 個 helper 統一邏輯：

9.1 四個 helper 函式

```
isDecisionOverdueAt(d, asOf = NOW): boolean  
isDecisionInProgressAt(d, asOf = NOW): boolean  
isDecisionCompletedAt(d, asOf = NOW): boolean  
daysOverdue(d, asOf = NOW): number
```

9.2 isDecisionOverdueAt 規則

- (1) 已決議 (`decidedAt ≤ asOf`)
- (2) `dueDate < asOf`（截止日已過）
- (3) 尚未完成 OR `completedAt > asOf`
- (4) 排除 `dueDate` 為 `NaN` / 「即時生效」字串 → 避免顯示「逾期 NaN 天」

9.3 統一使用點

`Dashboard`、`Decisions`、`NotificationPanel`、`MeetingPrep`、`WhatIf`、`orgHealth`、`computeORI` 全部改用 helper，不再依賴 `status` 字串。

好處：自動偵測「`dueDate` 過期但 `status` 仍是執行中」的決策，避免人為遺漏
用戶在 UI 新增決策若忘了改 `status`，系統會自動判定 →
不會出現「健康度被扣分但找不到對應決策」的詭異情境。

10. ★ Decision Impact 決策成效追蹤

對每筆「已完成」決策，比較決策前 vs 完成後 N 週的組織健康度，量化該決策對組織的影響。

10.1 計算邏輯

```
before = computeHealthSnapshot(decidedAt - 1 day)
afterWanted = completedAt + windowWeeks × 7
afterAsOf = min(afterWanted, NOW) # clamp ██████████
after = computeHealthSnapshot(afterAsOf)
deltaOverall = after.overall - before.overall
score = deltaOverall +  $\Sigma(\text{████████} \pm 2)$ 
```

10.2 評分判定

正面（綠）	$\text{score} \geq 3$
中性（灰）	$-3 < \text{score} < 3$
負面（紅）	$\text{score} \leq -3$
追蹤中（ ）	完成才不到 4 週 → 顯示「需累積 28 天才能算最終成效」

10.3 主管成效排行（Leader Scorecard）

依 decidedBy 分組（董事會 / 投資委員會 / 營運會議），計算每組的：

- 平均成效分（avgImpactScore）
- 正面 / 中性 / 負面決策計數

依平均分排序，第一名加 標籤。位於決策追蹤頁頂部。

為什麼有用？

管理層做完決策後，沒人告訴他效果如何 — 系統量化每筆決策對組織健康度的影響，形成「Plan → Decide → Track → Learn」閉環。

11. ★ What-if 決策模擬器

互動式 sandbox，讓管理層在做決策前先模擬後果。狀態用 React useState 管理一份 scenario 物件，套用後即時重算 computeHealthSnapshot，與現況雙圖層比對。

11.1 四個情境模組

解掉卡點	scenario.resolvedBlockerIds: Set, 被選中的卡點 status → resolved
加速逾期決策	scenario.expeditedDecisionIds, 立即 status → 已完成、completedAt = NOW
立即簽收交接	scenario.signedHandoffIds, 待簽收 → 已簽收、清除 hoursOverdue
增加部門人力	scenario.extraHeadcount: { [dept]: 0..5 }, 模擬支援人力擴充

11.2 雙圖層雷達比對

用 Recharts RadarChart 同時繪製：

- 灰色：現況 6 維健康度
- 紫色：套用 scenario 後的模擬快照

右側顯示整體分數對比（現況 → 模擬後）、delta 數字、各維度個別 delta。

11.3 智能建議

$\text{delta} \geq +5$	顯著改善 強烈建議執行
$\text{delta} +2 \sim +5$	有改善，可考慮執行
$\text{delta} -2 \sim +2$	影響不大，可保留資源
$\text{delta} -5 \sim -2$	略為惡化，需評估
$\text{delta} \leq -5$	顯著惡化 ⚠ 不建議執行

demo 場景

教授問：「如果你解掉田宮電機，會怎樣？」你（點一下）：「健康度從 54 跳到 62，+8.4 分。卡點健康從 跳到了 √。」— 演算法的價值瞬間具象化。

12. 資料同步機制與保護邏輯

採樂觀同步（Optimistic Sync）：UI 立即更新 → useEffect 監聽 → 寫回 Firestore。

12.1 SEED 保護條款（v2.1 強化）

reports / handoffs	< 10 筆 → 視為「首次使用」，用 SEED 填充；含舊「第 N 週」格式也重置
blockers / history / meetings	空時 fallback 到 SEED
departments / users （v2.1 新增）	空時也 fallback。避免 Firestore 集合誤刪導致 app 崩潰

13. Bug 修復記錄 (v2.0 → v2.1)

v2.0 → v2.1 修復 9 件 bug，涵蓋跨頁面數據一致性、邊界條件、SEED 保護：

B1	Dashboard analyzeBlockerRecord 漏傳 history → percentile / level 與其他頁不一致	補上第 3 個參數 history
B5	analyzeBlockerRecord 內部用 new Date() → 歷史快照 days 永遠偏大	新增 asOf 參數，預設 NOW；orgHealth + OrgHealthCard 都傳入
B2	各處 d.status === '逾期' vs orgHealth 動態判定，兩套並存	新增 4 helper (isDecisionOverdueAt / InProgressAt / CompletedAt / daysOverdue)，全系統統一
B3	dueDate 為 NaN 字串時 → 顯示「逾期 NaN 天」	helper 內部 isNaN 檢查 + clamp 0
B6	decisionImpact.afterAsOf 可能落在未來 → 「決策後快照 = 現況」	clamp 到 NOW，未滿 4 週顯示「追蹤中」
B8	orgHealth submissionRate 可超過 1（管理層交週報導致）	分母與分子皆過濾 expectedDeptSet，加 Math.min(1, ...)
B9	EmployeeLoad 過載百分比在空資料 → NaN%	加 loads.length > 0 守門
B7	Firestore departments / users 集合誤刪 → app 崩潰	兩個集合都加 SEED fallback
B11	Dashboard 未使用的 Sparkles import	移除

14. 結語：方法論選擇背後的设计取舍

本系统的核心设计理念是「用恰当的演算法解决恰当规模的问题」。我们不采用 LLM、Transformer，原因是：

1. 资料量少：53 笔历史案、240 笔週报、150 笔交接，BM25F 比 Embedding 更可靠且零成本。
2. 可解释性：管理层需要知道「为什么推荐这笔」，BM25F 可逐项列出命中词贡献。
3. 机密敏感：投资公司资料不能送 cloud API，本系统全部在前端 / 自有 Firebase。
4. 计算成本：BM25F、Gini、percentile 都是 $O(N)$ 或 $O(N \log N)$ ，在浏览器毫秒级完成。

14.1 演算法选用对照

資訊檢索	BM25F (Lucene 同款) — 而非 TF-IDF / cosine 或 sentence embedding
風險量化	Empirical Percentile — 而非絕對天數門檻
不平均度	Gini 係數 (經濟學標準) — 而非標準差
時間衰減	Exponential Decay 含 asOf 切片 — 支援歷史快照
拐點偵測	Local Minima — 而非機器學習 anomaly detection
組織網絡	Force-directed Graph + 雙向 Matrix
決策狀態	動態日期判定 helper — 不依賴可能過期的 status 字串
決策成效	前後快照對比 + 時間 clamp 避免未來

14.2 v2.1 的设计閉環

v2.1 形成完整的 Plan → Decide → Track → Learn 閉環：

Plan (規劃) · 卡點分析 + 員工負載 + 組織健康度 看現狀

Decide (決策) · What-if 模擬器 決策前看後果

Track (執行) · 決策追蹤 + 交接 + 週報

Learn (學習) · Decision Impact 量化效益、Leader 排行

下次做決策時，系統可用過去的 impact 推薦類似類型 — 形成自我強化的決策智慧。

「資料越少，演算法的選擇越重要。」

「跨頁面的一致性，比單頁的炫技更重要。」

—— 串連系統設計哲學